Chapter 7. Deadlocks. Exercises

Original problems. State assumptions, units, and intermediate reasoning. Answers are in the separate solutions PDF.

OS07-E01. Four necessary conditions

Two processes each hold one exclusive lock and wait for the other lock.

1. Identify the four necessary conditions concretely.
2. Does making one lock preemptible necessarily solve the whole system's deadlock problem?
3. Is a cycle sufficient if another instance can be released?

Check your approach. Necessary conditions organize the analysis; proof of deadlock must exclude the relevant progress transitions.

OS07-E02. Safe and unsafe states

Available=(2,1,0). Allocation is P1=(1,0,0), P2=(0,1,0), P3=(0,0,1). Need is P1=(1,1,0), P2=(2,1,0), P3=(3,0,0).

1. Find a safe completion order.
2. What is Work after P1 completes and returns its allocation?
3. Change P1's Need to (2,1,1). Validate Max against total resources and determine whether the changed state remains safe.

Check your approach. Check 0<=Allocation<=Max<=Total, then show every Work and Finish update.

OS07-E03. Victim selection

A resource is reclaimed while its owner has modified only half the nodes needed for a shared-tree update.

1. Why is merely releasing its lock unsafe?
2. Which recovery unit can restore a consistent state?
3. Is a limit of k consecutive selections per process enough to prevent starvation? Give a counterexample. Assume a fixed finite set of processes; others can be safely rolled back, release their resources, and be paused from requesting again in finite time. Each process's maximum demand is at most total resources; there are no external holders or new arrivals; resource grants and CPU scheduling are fair; and each process has finite remaining work. Design a protection rule and explain why every process eventually finishes.

Check your approach. Recovery integrity and victim fairness require separate arguments.

OS07-E04 Resource-graph cycle

R1 has two instances. P1 holds one and waits for unique R2. P2 holds R2 and waits for R1. P3 holds the other R1 and needs nothing else.

Instances are exclusive and nonpreemptible. Pending waits do not cancel and holdings are not released early. P3 is eventually scheduled and finishes in finite time, returning R1. Once their current requests are granted, P2 and P1 each finish in finite time and return all holdings.

1. List request and assignment edges.
2. Is there a cycle and do the four conditions appear?
3. Is the state deadlocked? Give a completion order.

OS07-E05 Banker request

T0 Total=(10,5,7), Available=(3,3,2). P0…P4 Allocation=(0,1,0),(2,0,0),(3,0,2),(2,1,1),(0,0,2); Need=(7,4,3),(1,2,2),(6,0,0),(0,1,1),(4,3,1). P1 requests(1,0,2).

1. Check claim and availability componentwise.
2. Give trial Available, Allocation1, Need1.
3. Show every safety Work row and a safe sequence.
4. What must happen if a trial is unsafe?

OS07-E06 Multi-instance detection and recovery

Total=(7,2,6), Available=(0,0,0). P0…P4 Allocation=(0,1,0),(2,0,0),(3,0,3),(2,1,1),(0,0,2); Request=(0,0,0),(2,0,2),(0,0,1),(1,0,0),(0,0,2).

Use a fixed-snapshot model: pending requests are neither canceled nor reduced, resources are nonpreemptible and are not released early. Each surviving process, once its current request is granted, finishes in finite time, returns all holdings, and makes no further requests. P0's zero request means it can continue. If P1 is chosen for recovery, assume it can be safely terminated and its holdings reclaimed in finite time. Conclusions under this model do not guarantee actual behavior outside its assumptions.

1. Show Work, feasible process, and Finish at every reduction.
2. Identify the final deadlocked set.
3. Abort P1. Show returned resources and remaining reduction.
4. Why does recovery require rollback as well as reclaiming locks?

OS07-E07 One cycle, two outcomes

R1 has two instances and R2 one; vectors use (R1,R2). Initially Available=(0,0). Allocations for P1/P2/P3 are (1,0), (0,1), (1,0); current requests are (0,1), (1,0), (0,0). After its current request is satisfied, each process can finish and return all holdings. There is no early release, preemption, or cancellation.

1. Draw instances, assignment edges, and request edges. Identify a cycle. Does this cycle alone establish deadlock?
2. Reduce from Work=(0,0), giving every Request≤Work test, returned allocation, and new Work. Must live Available equal (2,1) when this simulation ends?
3. Execute that completion order and list Available after each grant and actual return. Include the deductions made by grants.
4. Change only P3's Request to (0,1). Show all three failed vector comparisons, identify the deadlocked set, and name the added edge.

OS07-E08 Trials, rollback, and actual completion

Vectors are (A,B). Total=(4,3), Available=(1,1). Allocation for P0/P1/P2 is (1,0), (1,1), (1,1); Max is (3,2), (2,2), (3,2). Resources are nonpreemptible, returned together at completion, and Max is an upper bound.

(1) Independently trial P2's (0,1) and P1's (0,1). Give Need, every Work/Finish state and the result. Do not chain the trials.
(2) Specify all three rollback updates for the rejected request. How does restoring only Available violate conservation?
(3) Consider counterfactual U after P0 receives (1,0). Would one extra A or one extra B make U safe? Give an order or every candidate's failing component.
(4) Without extra resources, let P0/P1/P2 request only (0,1), (1,0), (1,0), respectively, then finish. List actual Available after every grant and return. Why is this actual completion order not a safe sequence for U?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 7.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study